Lab Experiments

11 Write the python program for Map Coloring to implement CSP.

Aim

To implement the Map Coloring problem using a Constraint Satisfaction Problem (CSP) and ensure that adjacent regions have different colors.

Algorithm

1. Define the regions and their neighboring regions.
2. Define the available colors.
3. Select an uncolored region.
4. Assign a color that does not conflict with neighboring regions.
5. Use backtracking if no valid color is available.
6. Display the final color assignment.

Program:

```
# Map Coloring using CSP

regions = ['A', 'B', 'C', 'D']
colors = ['Red', 'Green', 'Blue']

neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

assignment = {}
```

```
def is_safe(region, color):
    for neighbor in neighbors[region]:
        if assignment.get(neighbor) == color:
            return False
    return True

def solve():
    if len(assignment) == len(regions):
        return True

    for region in regions:
        if region not in assignment:
            for color in colors:
                if is_safe(region, color):
                    assignment[region] = color

                    if solve():
                        return True

                    del assignment[region]
            return False

    return True

if solve():
    print("Map Coloring Solution:")
    for region in regions:
        print(region, ":", assignment[region])
else:
    print("No solution exists")
```

Output:

```
>>> ================================ RESTART ================================
>>>
Map Coloring Solution:
('A', ':', 'Red')
('B', ':', 'Green')
('C', ':', 'Blue')
('D', ':', 'Red')
>>>
```

Result:

The Map Coloring problem was solved successfully using CSP and backtracking.

2. Write the python program for Tic Tac Toe game

Aim

To develop a simple Tic-Tac-Toe game for two players using Python.

Algorithm

1. Create a 3 × 3 board.
2. Display the board.
3. Allow players X and O to enter positions from 1 to 9.
4. Check whether the position is empty.
5. Check for a winning combination or a draw.
6. Repeat until the game ends.

Program:

```
# Tic-Tac-Toe for two players
board = [' '] * 9
def display():
    print()
    for i in range(0, 9, 3):
        print(" " + " | ".join(board[i:i+3]))
```

```
        if i < 6:
            print("---+---+---")
    print()
def winner():
    lines = [
        (0, 1, 2), (3, 4, 5), (6, 7, 8),
        (0, 3, 6), (1, 4, 7), (2, 5, 8),
        (0, 4, 8), (2, 4, 6)
    ]
    for a, b, c in lines:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]
    return None
player = 'X'
while True:
    display()
    try:
        pos = int(input("Player " + player +
                        ", enter position (1-9): ")) - 1
    except ValueError:
        print("Enter a number.")
        continue
    if pos < 0 or pos > 8 or board[pos] != ' ':
        print("Invalid position. Try again.")
        continue
    board[pos] = player
    if winner():
        display()
        print("Player", player, "wins!")
        break
    if ' ' not in board:
        display()
        print("Game is a draw!")
        break
```

```
    player = 'O' if player == 'X' else 'X'
```

Output:

```
>>>
()
   |   |
---+---+---
   |   |
---+---+---
   |   |
()
Player X, enter position (1-9):
```

Result:

The Tic-Tac-Toe game was implemented successfully with turn-based play, win detection, and draw detection.

3 Write the python program to implement Minimax algorithm for gaming

Aim

To implement the Minimax algorithm to find the best possible move in a two-player game.

Algorithm

1. Represent the game as a tree of possible moves.
2. Assign scores to terminal states.
3. The maximizing player selects the maximum score.
4. The minimizing player selects the minimum score.
5. Return the best score and move.

Program:

```
# Minimax algorithm using a game tree
tree = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G']
}
scores = {
    'D': 3, 'E': 5,
    'F': 2, 'G': 9
}
def minimax(node, maximizing):
    if node in scores:
        return scores[node]
    values = []
    for child in tree[node]:
        values.append(minimax(child, not maximizing))
    if maximizing:
        return max(values)
    else:
        return min(values)
best_score = minimax('A', True)
print("Best score:", best_score)
```

Output:

```
>>> ================================ RESTART ================================
>>>
('Best score:', 3)
>>>
```

Result:

The Minimax algorithm was implemented successfully to select the best score for the maximizing player.

4. Alpha-Beta Pruning Algorithm (CO3)

Aim

To implement Alpha-Beta Pruning to optimize the Minimax algorithm by eliminating branches that cannot affect the final decision.

Algorithm

1. Initialize alpha to negative infinity and beta to positive infinity.
2. Evaluate the game tree recursively.
3. The maximizing player updates alpha.
4. The minimizing player updates beta.
5. Stop exploring a branch when alpha is greater than or equal to beta.
6. Return the optimal score.

Program:

```
# Alpha-Beta Pruning

tree = {

   'A': ['B', 'C'],

   'B': ['D', 'E'],

   'C': ['F', 'G']

}

scores = {

   'D': 3, 'E': 5,

   'F': 2, 'G': 9

}
```

```
def alphabeta(node, alpha, beta, maximizing):
    if node in scores:
        return scores[node]
    if maximizing:
        value = float('-inf')
        for child in tree[node]:
            value = max(value,
                        alphabeta(child, alpha, beta, False))
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value = float('inf')
        for child in tree[node]:
            value = min(value,
                        alphabeta(child, alpha, beta, True))
            beta = min(beta, value)
            if alpha >= beta:
                break
```

```
        return value

result = alphabeta('A', float('-inf'),
                   float('inf'), True)

print("Best score:", result)
```

Output:

```
>>> ================================ RESTART ================================
>>>
('Best score:', 3)
>>>
```

Result:

Alpha-Beta Pruning was implemented successfully and reduces unnecessary game-tree exploration while preserving the Minimax result.

5. Decision Tree (CO4)

Aim

To implement a simple Decision Tree classifier to predict whether a student passes based on study hours.

Algorithm

1. Prepare training data containing study hours and pass/fail labels.
2. Select a threshold to divide the data.
3. Create decision branches based on the threshold.
4. Accept study hours as input.
5. Predict whether the student passes or fails.

Program:

```
# Simple Decision Tree Classifier

def predict(hours):
    if hours >= 5:
        return "Pass"
    else:
        return "Fail"

print("Decision Tree: Student Result Prediction")

hours = float(input("Enter study hours: "))

print("Prediction:", predict(hours))
```

Output:

```
>>> ================================ RESTART ================================
>>>
Decision Tree: Student Result Prediction
Enter study hours: 5
('Prediction:', 'Pass')
>>>
```

Result: A simple Decision Tree classifier was implemented successfully to predict the student's result based on study hours.

6. Feed Forward Neural Network (CO4)

Aim

To implement a simple Feed Forward Neural Network using Python to demonstrate how input data passes through a hidden layer to produce an output.

Algorithm

1. Initialize input values and weights.
2. Calculate the weighted sum at the hidden neuron.
3. Apply the sigmoid activation function.
4. Calculate the weighted sum at the output neuron.
5. Apply the sigmoid function again.
6. Display the predicted output.

Program:

```
# Feed Forward Neural Network

# One input, one hidden neuron, one output

import math

def sigmoid(x):

    return 1 / (1 + math.exp(-x))

# Input value

x = float(input("Enter input value (0 to 1): "))

# Weights and biases

w1 = 0.5
```

```
b1 = 0.1

w2 = 0.8

b2 = 0.2

# Hidden layer

hidden = sigmoid(x * w1 + b1)

# Output layer

output = sigmoid(hidden * w2 + b2)

print("Hidden layer output:", round(hidden, 4))

print("Final network output:", round(output, 4))
```

Output:

```
>>> ================================ RESTART ================================
>>>
Enter input value (0 to 1): 5
('Hidden layer output:', 0.9309)
('Final network output:', 0.72)
>>>
```

Result:

A basic Feed Forward Neural Network was implemented successfully using weighted sums and the sigmoid activation function.